

Day 26: Logistic Regression (Binary Classification Engine)

COMPACT WRAPPERS, INTERACTIVE TERMINALS & SKLEARN OPTIMIZATION

This technical reference guide covers two comprehensive implementations of a Logistic Regression workflow built inside Jupyter Notebook cells. It explores scratch-level computational loops, native mathematical gradient derivations, production-ready live predictive loops with persistent UI error handling, and high-level industrial workflows using modern Scikit-Learn integrations.

Approach 1: Scratch Training Loop & Fail-Safe Live Terminal

This approach runs an independent optimization process using basic mathematical principles to arrive at stable model parameters, freezing weights instantly into memory variables without requiring high-level packages.

```
import math
from IPython.display import clear_output

# 1. TRAINING PHASE (Engine sirf ek baar chalega)
X_train, Y_train, m, c, learning_rate, purana_loss = 2.0, 1.0, 1.5, -2.0, 0.1, 999999.0

while True:
    Z = (m * X_train) + c
    P = 1 / (1 + math.exp(-Z))
    loss = - (Y_train * math.log(P) + (1 - Y_train) * math.log(1 - P))

    if abs(purana_loss - loss) < 0.000001:
        break

    m = m - (learning_rate * (X_train * (P - Y_train)))
    c = c - (learning_rate * (P - Y_train))
    purana_loss = loss

real_m, real_c = m, c

# 2. UPDATED LIVE TERMINAL (Alphabet safe with persistent message)
default_X, error_msg = 2.5, ""

while True:
    clear_output(wait=True)
    print("✅ ENGINE TRAINED SUCCESSFULLY!")
    print(f"Locked Parameters -> Real m: {real_m:.4f} | Real c: {real_c:.4f}\n" + "="*55)
    print("👉 LIVE SALES PREDICTION ENGINE (Type 'exit' to stop) 👉\n" + "="*55)

    Z_naya = (real_m * default_X) + real_c
    P_naya = 1 / (1 + math.exp(-Z_naya))

    print(f"Last Valid Input Processed (X): {default_X}")
    print(f"Formula Output      -> Z = ({real_m:.4f} * {default_X}) + ({real_c:.4f}) = {Z_naya:.4f}")
    print(f"Khareedne ka chance   : {P_naya * 100:.2f}%")
    print(f"🟢 FINAL ANSWER       : HAAN" if P_naya >= 0.5 else "🔴 FINAL ANSWER       : NAA")
    print("-" * 55)

    if error_msg:
```

```

print(error_msg + "\n" + "-"*55)

user_input = input("Bhai, naya Score (X) dalo aur Enter dabao: ").strip()
if user_input.lower() == 'exit':
    print("\nPrediction engine band ho gaya. Shukriya bhai!")
    break

try:
    default_X = float(user_input)
    error_msg = ""
except ValueError:
    error_msg = f"❌ Kripya sahi number enter karein! ('{user_input}' ek number nahi hai)"

```

Core Logic Mechanics:

- **Gradient Descent Optimization:** Updates parameters using partial derivatives until variance falls beneath 10^{-6} .
- **IPython Display Management:** `clear_output(wait=True)` prevents flickering inside live loops.
- **Persistent Error Flashes:** Captures alphabet errors cleanly into the UI status state without disrupting terminal continuity.

Approach 2: Production Sklearn Automation Workflow

This implementation delegates background gradient convergence to industry-standard packages, replacing entire manually computed mathematical pipelines with optimized solver algorithms.

```

import numpy as np
from sklearn.linear_model import LogisticRegression
from IPython.display import clear_output

# 1. DATA INPUT (Minimum 2 distinct baseline target classes required)
X_train = np.array([[2.0], [0.5]])
Y_train = np.array([1, 0])

# 2. SEAMLESS AUTOMATED TRAINING
model = LogisticRegression().fit(X_train, Y_train)

print("✅ MODEL TRAINED AUTOMATICALLY BY SKLEARN!")
print(f"Auto-Calculated -> m: {model.coef_[0][0]:.4f} | c: {model.intercept_[0]:.4f}\n" +
      "-"*55)

# 3. LIVE PREDICTION ENGINE
while True:
    clear_output(wait=True)
    inp = input("\nBhai, bas naya Score (X) dalo [ya 'exit']: ").strip()
    if inp.lower() == 'exit': break

    try:
        naya_X = float(inp)

```

```

probability = model.predict_proba([[naya_X]])[0][1] * 100
prediction = model.predict([[naya_X]])[0]

print(f"-> Khareedne ka chance: {probability:.2f}%")
print("🟢 FINAL ANSWER: HAAN" if prediction == 1 else "🔴 FINAL ANSWER: NAA")

except ValueError:
    print("❌ Kripya sahi number enter karein!")

```

Critical Architectural Rule (Solver Class Exception Solution):

Attempting to call Scikit-Learn estimators using a singular classification state throws a fatal `ValueError`. Classification requires structural boundaries; therefore, training matrices must explicitly balance negative reference indices (0) against target states (1).

Deep-Dive Token Parsing Mechanics:

- `model.predict_proba(...)[0][1]`: Accesses row 0 and parses the exact class-1 (positive) probability output directly.
- `model.predict(...)[0]`: Extracts the raw scalar code value directly out of the generated prediction bracket wrapper structure.